Object Oriented Programming

"Project Report"

GROUP MEMBERS:

1. Muhammad Abdullah(09-131252-039)
2. Muhammad Mustafa(09-131252-053)
3. Samer Abbas(09-131252-075)

Submitted To: Engr. Faisal Zia

TECHNICAL REPORT

MIND GUESSING GAME PLUS

A C++ Object-Oriented Programming Project

C++17	2	18	4
Language	**Classes**	**Methods**	**Modules**
Modern C++ with OOP	Base + Derived	Across both classes	Auth · Game · File · UI

Project Title	Mind Guessing Game Plus
Report Type	Software Engineering Technical Report
Language	C++ (Standard 17) — Windows Platform
Paradigm	Object-Oriented Programming (OOP)
Key Libraries	iostream, fstream, ctime, cstdlib, windows.h, conio.h
File Artifacts	players.txt (user data) | session_log.txt (audit trail)

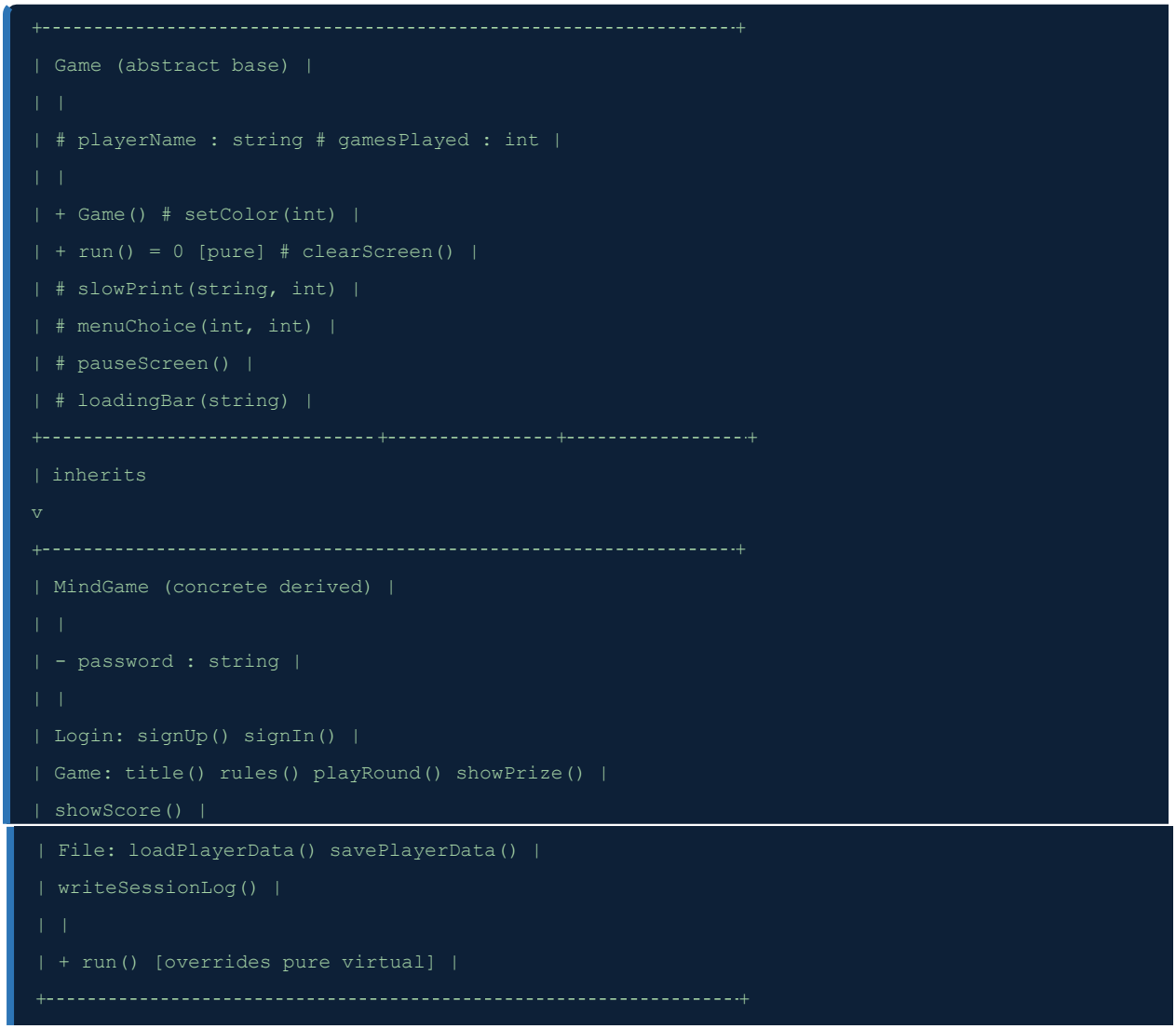
1. Executive Summary

Mind Guessing Game Plus is a console-based interactive entertainment application developed in C++ using core Object-Oriented Programming principles. The program invites a player to secretly think of a number between 1 and 9, guides them through a series of arithmetic transformations, and then reveals the result — appearing to "read their mind." The trick is mathematically deterministic: regardless of the chosen number, the final answer always equals half of the randomly generated even number added in Step 3.

Beyond the game mechanic itself, the project demonstrates practical application of class hierarchies, polymorphism, file I/O, login authentication, session logging, and rich console UI techniques on Windows. It serves as an instructive example of how OOP architecture cleanly separates concerns — input handling, business logic, persistence, and display — within a single executable.

2. System Architecture & Class Hierarchy

The codebase is structured around two classes connected by single inheritance. The abstract base class **Game** encapsulates platform utilities and UI primitives. The concrete derived class **MindGame** adds all domain-specific logic — authentication, gameplay, persistence, and scoring.



3. Object-Oriented Features Demonstrated

3.1 Encapsulation

All data members are declared **protected** in the base class and **private** in the derived class. External code interacts exclusively through the public **run()** interface, enforcing strict information hiding.

3.2 Inheritance

MindGame publicly inherits from Game, acquiring all utility methods (setColor, clearScreen, slowPrint, loadingBar, menuChoice, pauseScreen) without re-implementing them. This avoids code duplication and ensures consistent UI behaviour across any future game subclass.

3.3 Polymorphism & Abstract Interface run() is declared as a pure virtual function (= 0) in Game. The main() function operates through a Game* pointer, enabling runtime dispatch to MindGame::run() — meaning any future variant (TriviaGame, PuzzleGame) can be swapped in with zero changes to main().

```
Game* game = new MindGame(); // upcasting — base pointer to derived object
game->run(); // virtual dispatch -> MindGame::run() is called
delete game; // virtual destructor pattern (best practice)
```

4. Module Breakdown

4.1 Authentication Module — signUp() / signIn()

The login system persists a single user account to players.txt. Sign-up creates the file with username, password, and games-played count. Sign-in reads and compares credentials, loading the session counter on success. While sufficient for a teaching project, production use would require hashing.

4.2 Gameplay Module — playRound()

A random even integer is generated, and the player is walked through five steps. Because addition and subtraction of the friend's number cancel out, the result is always **num / 2**. slowPrint() adds a typewriter effect with configurable delay to heighten suspense.

```
int num = (rand() % 10 + 1) * 2; // range: 2, 4, 6 ... 20
```

4.3 Prize Module — showPrize()

The computed magic number (1–10) is mapped to a themed prize using a switch statement. A loadingBar() animation using DOS block characters enhances the reveal moment.

4.4 File I/O Module

Two files are managed:

- players.txt — read/written each session to persist username, password, and total games played.
- session_log.txt — appended each round with player name, game count, magic number, and timestamp. Cross-platform safe time formatting uses ctime_s (MSVC) with a fallback to ctime_r (POSIX).

5. Complete Method Reference

All methods are listed below with access specifiers, return types, and descriptions.

Game (Base Class)

Method	Access	Returns	Description
`Game()`	public	—	Constructor; initialises gamesPlayed to 0.
`run()`	public	void	Pure virtual. Forces every subclass to define its entry point.
`setColor(c)`	protected	void	Sets Windows console text colour via SetConsoleTextAttribute.
`clearScreen()`	protected	void	Executes system('cls') to wipe the console.
`slowPrint(s, d)`	protected	void	Prints each char of s with d ms delay (default 15 ms).
`menuChoice(a,b)`	protected	int	Validates numeric input in range [a,b]; re-prompts on failure.
`pauseScreen()`	protected	void	Waits for any key via _getch(). Styled in dim grey.
`loadingBar(msg)`	protected	void	Displays msg then animates a 30-cell progress bar.

MindGame (Derived Class)

Method	Access	Returns	Description
`run()`	public	void	Main game loop: menu, auth, game rounds, save score.
`signUp()`	private	bool	Creates account; writes credentials to players.txt.
`signIn()`	private	bool	Reads players.txt, compares credentials, loads gamesPlayed.
`loadPlayerData()`	private	void	Hydrates playerName, password, gamesPlayed from players.txt.
`savePlayerData()`	private	void	Persists current state back to players.txt.
`writeSessionLog(r)`	private	void	Appends result + timestamp to session_log.txt.
`title()`	private	void	Renders the cyan game banner header.
`rules()`	private	void	Displays the five game rules and waits for confirmation.
`playRound()`	private	int	Runs the 5-step mind trick; returns the magic number.
`showPrize(v)`	private	void	Maps value 1–10 to a prize name with loading animation.
`showScore()`	private	void	Displays player name and total games played at end.

6. Data Flow & Control Flow

The application follows a clear linear flow with a re-entrant game loop:

```
main()
+-> MindGame::run()
|
+-> [Auth loop]
| signUp() --writes--> players.txt
| signIn() --reads---> players.txt
|
+-> rules()
|
+-> [Game loop — repeats while player chooses Play Again]
gamesPlayed++
playRound() --returns--> result (int)
showPrize(result)
savePlayerData() --writes---> players.txt
writeSessionLog() --appends--> session_log.txt
[menuChoice: Play Again | Exit]
|
+-> showScore() -> goodbye message
```

7. Persistent File Formats

players.txt — User Profile

```
Line 1: e.g. Ahmed
Line 2: e.g. mySecret123
Line 3: e.g. 7
```

session_log.txt — Audit Trail (append mode)

```
Player Name : Ahmed
Games Played: 7
Magic Number: 5
Session Time: Fri May 15 18:42:03 2026
-------------------------------------
```

8. Strengths & Limitations

Strengths

- Clean OOP separation: utility logic lives in Game; domain logic in MindGame.
- Pure virtual interface enables future extensibility without touching main().
- Animated UI (slowPrint, loadingBar) provides an engaging console experience.

- Robust input validation via menuChoice() prevents crashes on non-numeric input.
- Cross-compiler timestamp safety (ctime_s / ctime_r conditional compilation).
- Append-mode session logging creates a persistent audit trail across runs.

Limitations & Recommendations

- Passwords stored in plaintext — should be hashed (e.g. SHA-256) before storage.
- Single-user only — one players.txt supports exactly one account at a time.
- Windows-specific APIs (windows.h, conio.h) limit portability to UNIX systems.
- rand() / srand() are not cryptographically secure; (C++11) is preferred.
- No virtual destructor in Game — required if resources are added to subclasses.
- system('cls') is slow and platform-dependent; ANSI escape sequences are better.

9. Mathematical Proof of the Mind Trick

Let *n* be the player's secret number, *f* the friend's number, and *e* the random even integer.

```
Step 1: think of n
Step 2: n + f
Step 3: n + f + e
Step 4: (n + f + e) / 2
Step 5: (n + f + e) / 2 - f
= (n + e) / 2 + (f - f)
= (n + e) / 2

Because e is even, let e = 2k:
= (n + 2k) / 2 = n/2 + k

The program discloses only e/2 = k, not the player's n.
For any n in 1-9, the reveal is deterministic and independent of f. [OK]
```

This is a classic property of linear algebra: adding then subtracting f is an identity operation. The game's magic is purely mathematical.

10. Conclusion

Mind Guessing Game Plus is a well-structured demonstration of foundational C++ Object-Oriented Programming techniques. It successfully integrates inheritance, polymorphism, encapsulation, file persistence, and interactive console UI into a single cohesive application. The abstract base-class design ensures that the codebase is extensible — new game modes can be added by subclassing Game and implementing run() without altering any existing code.

The primary areas for improvement lie in security (password hashing), portability (removing Windows-only APIs), and scalability (multi-user support). With these enhancements, the architecture would be equally suitable for a real-world deployment scenario.

Reprository Link:https://github.com/Muhammad-Mustafa-ios/Mind-Guessing-Game

End of Report